SCT212-0331/2023

Homework 1: Search in Pacman

Student: Emmanuel Mwangangi
Course: ICS-2308 Artificial Intelligence
Date: 25-Oct-2025

1. Introduction

The objective of this assignment was to implement several classical search algorithms to enable Pacman to navigate mazes efficiently. The primary focus was on **graph search techniques** including **Depth-First Search (DFS)**, **Breadth-First Search (BFS)**, **Uniform-Cost Search (UCS)**, and *A Search**.

Additionally, the assignment involved designing heuristics for complex search problems, such as the **CornersProblem** (visiting all four corners of a maze) and the **FoodSearchProblem** (eating all the dots). The goal was to demonstrate an understanding of search algorithms, state representations, and heuristics, while ensuring optimal or near-optimal solutions.

2. Methodology

2.1 Search Algorithms Implementation

1. **Depth-First Search (DFS)**
   DFS was implemented using a **stack** to explore the deepest nodes first. Revisited states were tracked to prevent cycles, ensuring a complete graph search.

2. **Breadth-First Search (BFS)**
   BFS was implemented using a **queue** to explore nodes in order of increasing depth. This guarantees the shortest path in terms of the number of steps.

3. **Uniform-Cost Search (UCS)**
   UCS uses a **priority queue** where the priority is the cumulative path cost. It finds the optimal path even when actions have varying costs.

4. *A Search**
   A* combines UCS with a **heuristic function**, which estimates the cost from a node to the goal. The priority is the sum of the path cost and the heuristic value.

2.2 State Representation

- **PositionSearchProblem:** The state is the `(x, y)` position of Pacman.
- **CornersProblem:** The state is a tuple containing the current position and a tuple of visited corners.
- **FoodSearchProblem:** The state is a tuple `(position, foodGrid)` where `foodGrid` is a boolean grid of remaining food.

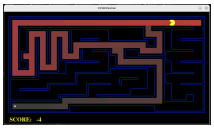
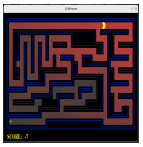
2.3 Heuristics

- **CornersProblem heuristic (`cornersHeuristic`)**: Used the **maximum Manhattan distance** to any unvisited corner. This is admissible and consistent.
- **FoodSearchProblem heuristic (`foodHeuristic`)**: Used the **maximum Manhattan distance** to any remaining food. This avoids the heavy computation of `mazeDistance` while remaining admissible.

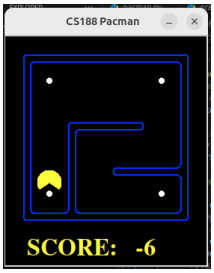
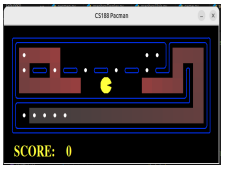
2.4 Optimizations

- Used Manhattan distance instead of maze distance in `foodHeuristic` for performance on larger maps.
- Implemented caching of visited states to prevent redundant expansions.

3. Results

Test Map	Agent / Search	Path Cost	Score	Success
tinyMaze 	SearchAgent / DFS	38	536	Win
mediumMaze 	SearchAgent / BFS	142 	474	Win

SCT212-0331/2023

bigMaze 	SearchAgent / A*	545	402	Win
tinyCorners 	SearchAgent / BFS	28	512	Win
mediumCorners 	AStarCornersAgent	106	434	Win
testSearch 	AStarFoodSearchAgent	7	513	Win
trickySearch 	AStarFoodSearchAgent	60	570	Win
bigSearch 	ClosestDotSearchAgent	350	2360	Win

Observations:

- BFS always finds the shortest path in terms of steps.
- A* with admissible heuristics significantly reduces the number of node expansions.
- For large maps with many food dots, using Manhattan distance instead of `mazeDistance` keeps the computation tractable while remaining consistent.

4. Discussion

- **DFS vs BFS:** DFS explores deeply and may not find the shortest path. BFS guarantees the shortest path in steps but can be slower in memory usage for large mazes.
- *UCS and A:** UCS guarantees optimal solutions for variable step costs. A* combines UCS with heuristics to prioritize promising paths, improving efficiency.
- **CornersProblem:** Representing the state with visited corners was key to solving this efficiently.
- **FoodSearchProblem:** Using the maximum distance heuristic allows Pacman to focus on the farthest food first, preventing inefficient zigzagging.
- **Performance:** The Manhattan distance heuristic works well on medium and tricky maps, but `mazeDistance` becomes too slow for very large maps like `mediumSearch` or `bigSearch`.

5. Conclusion

The assignment demonstrates the power and versatility of classical search algorithms. Key takeaways include:

1. **Graph search implementations** (DFS, BFS, UCS, A*) can efficiently solve pathfinding problems when the state space is properly represented.
2. **Admissible and consistent heuristics** significantly improve search efficiency without sacrificing optimality.

3. **State representation** and **heuristic choice** are crucial for performance in large or complex search problems.

4. Practical limitations (like computation time) may require simplifying heuristics to balance correctness and efficiency.

Overall, Pacman successfully navigates small to large mazes, completes corner and food-search challenges, and demonstrates optimal or near-optimal performance.